



UEA

UNIVERSIDAD
ESTATAL AMAZÓNICA

DESARROLLO DE APLICACIONES WEB

Unidad 2

Desarrollo del Lado del Cliente

Tema 2.1: Integración de frontend

Subtema 2.1.2: Uso de plantillas para renderizar contenido dinámico

DESARROLLO DE APLICACIONES WEB



Transformamos el mundo desde la Amazonía

Ing. Walter Rodrigo Núñez Zamora, Mgs.

DOCENTE - PERSONAL ACADÉMICO NO TITULAR OCASIONAL

UNIVERSIDAD ESTATAL AMAZÓNICA



SEMANA

7

DESARROLLO DE LA SEMANA 7

DEL LUN. 19 AL DOM. 25 DE ENERO / 2026

🎯 Resultado de aprendizaje: Aplicar técnicas de diseño de interfaces y desarrollo frontend y backend para implementar funcionalidades dinámicas y gestionar datos de manera eficiente.

☰ CONTENIDOS

📖 UNIDAD II : Desarrollo del lado del Cliente

- Tema 2.1: Integración de frontend
- Subtema 2.2.1: Uso de plantillas para renderizar contenido dinámico.



Subtema 2.1 " Uso de plantillas para renderizar contenido dinámico."

¿Qué es Uso de plantillas para renderizar contenido? dinámico

Es una técnica empleada en desarrollo web que permite generar páginas o vistas personalizadas combinando contenido estático (como HTML) con datos dinámicos proporcionados por el backend o el cliente.



¿Qué son las plantillas?

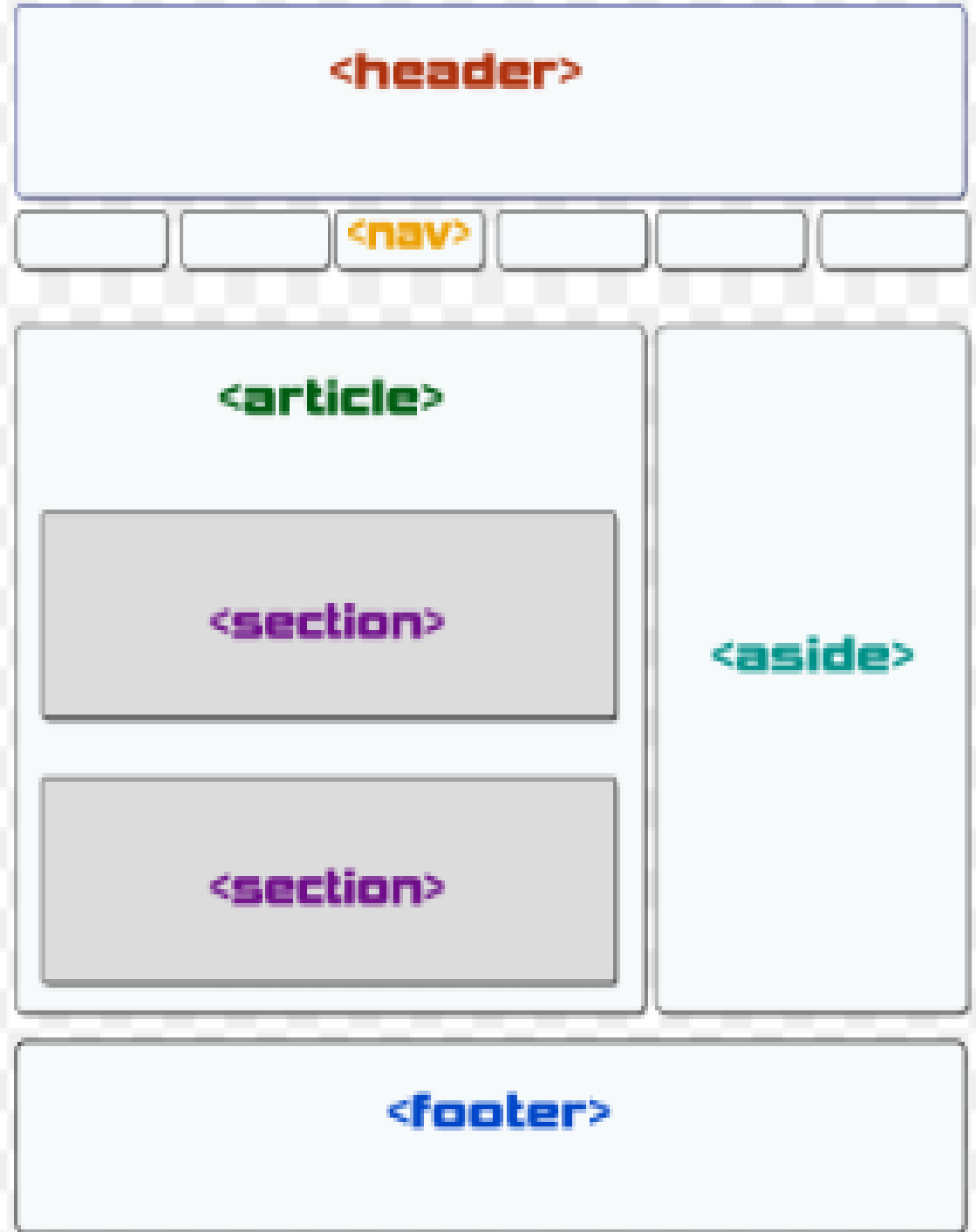
Una plantilla es un archivo que define el diseño y la estructura base de una página web, con marcadores o delimitadores que son reemplazados por datos específicos en tiempo de ejecución. Estos marcadores actúan como "lugares reservados" que se llenan con información variable.

Por ejemplo:

Html:

```
<h1>Bienvenido, {{ nombre_usuario }}</h1>
<p>Tienes {{ mensajes_no_leídos }} mensajes nuevos.</p>
```

Cuando se renderiza esta plantilla, los marcadores {{ nombre_usuario }} y {{ mensajes_no_leídos }} son reemplazados por datos reales proporcionados por el servidor



Cómo funcionan: Un motor de plantillas toma un archivo de plantilla y los datos proporcionados (como objetos, listas o valores), y genera un documento final, como HTML o cualquier otro formato, con el contenido actualizado.

Ventajas:

Separación de preocupaciones: Separa la lógica de la presentación, lo que hace que el código sea más limpio y fácil de mantener.

Reutilización: Las plantillas permiten reutilizar bloques de código de diseño sin repetirlo.

Facilita la personalización: Los datos pueden cambiar sin alterar la estructura de la página.



Motores de Plantillas Populares:

Jinja2 (Python): Usado comúnmente en frameworks como Flask y Django. Permite inyectar variables en el HTML y controlar la estructura de las páginas.

Handlebars (JavaScript): Motor de plantillas ligero y sencillo. Se utiliza en aplicaciones JavaScript.

Mustache (varios lenguajes): Similar a Handlebars, funciona en una variedad de lenguajes.

EJS (JavaScript): Otro motor para generar HTML en Node.js, con sintaxis similar a la de JavaScript.

Ejemplo con Jinja2 (Python):

```
from jinja2 import Template
template = Template('Hola {{ name }}!')
print(template.render(name='Juan'))
```

Este ejemplo crea un mensaje dinámico con el nombre "Juan", reemplazando la variable name en la plantilla.



Estructura básica de una plantilla

La estructura básica de una plantilla depende del motor de plantillas que estés utilizando, pero todas suelen compartir ciertos elementos esenciales: marcadores de variables, estructuras de control (como bucles y condicionales) y directivas para incluir o extender otras plantillas.

Estructura básica de una plantilla utilizando algunos motores populares:

a) Variables dinámicas

Son marcadores que serán reemplazados por valores al renderizar la plantilla.

b) Bloques de control

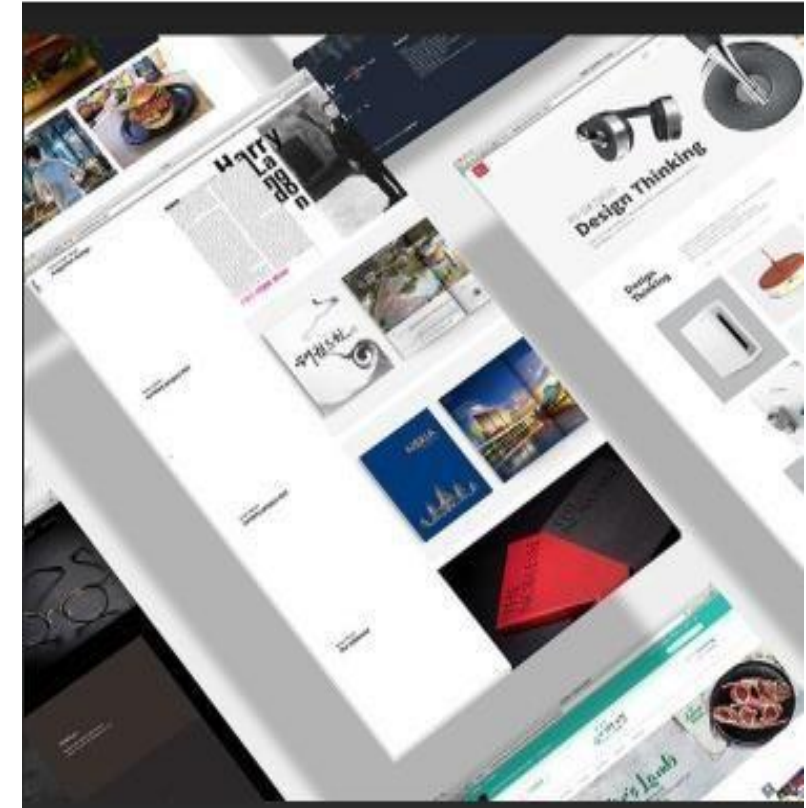
Bucle: Para iterar sobre listas o colecciones.

Condicional: Para mostrar contenido basado en condiciones.

c) Extensión e inclusión

Extender otra plantilla (usualmente para manejar layouts):

Incluir fragmentos reutilizables (como una barra de navegación):



Uso de Variables en Motores de Plantillas



1. Jinja2 (Python/Flask/Django)

Las variables se representan con doble llave `{{ }}` y se pasan al renderizar.

2. EJS (Node.js + Express)

Las variables se incluyen con `<%= variable %>`

3. Handlebars (Node.js + Express)

Las variables se colocan entre `{{ }}`.

4. JSX (React)

En React, las variables se usan directamente con llaves `{}`.

Alcance y Transformación de Variables

Filtros o Transformaciones (Ejemplo en Jinja2)

Puedes aplicar filtros a las variables en el motor de plantillas. Por ejemplo: `html`

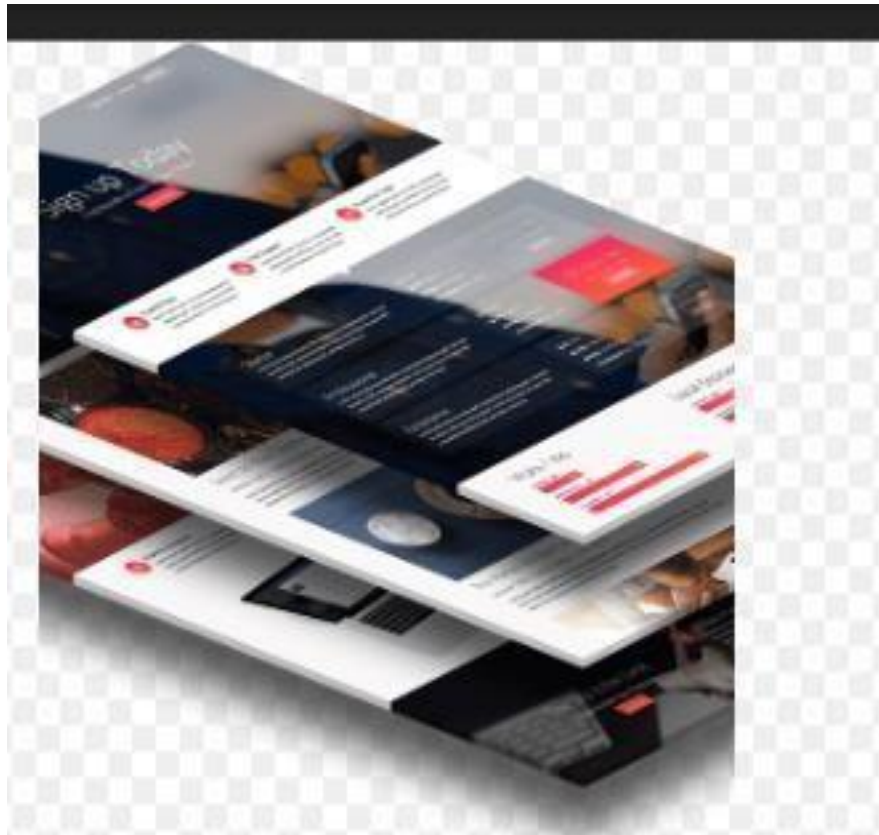
```
<p>Nombre en mayúsculas: {{ nombre|upper }}</p> <p>Edad más 10 años: {{ edad + 10 }}</p>
```

Escapado de Caracteres Especiales

En muchos motores de plantillas, los valores se escapan por defecto para evitar ataques de tipo XSS. Si necesitas insertar HTML sin escaparlo:

- Jinja2: Usa `|safe`.

Ciclo de vida de una plantilla



El ciclo de vida de una plantilla describe el proceso por el cual una plantilla es creada, utilizada, y renderizada en una aplicación dinámica. Este ciclo involucra la interacción entre el backend, el motor de plantillas y el cliente que consume el contenido final. Aquí tienes los pasos generales:

1. Creación de la plantilla

Descripción:

Se define la estructura básica del contenido dinámico.

Se utilizan marcadores para las variables, estructuras de control (condicionales, bucles), y directivas para incluir o extender otras plantillas.

2. Configuración en el servidor

Descripción:

El servidor (backend) se configura para utilizar un motor de plantillas. Se define el directorio donde se almacenan las plantillas.

Se envían datos dinámicos a la plantilla para que sean renderizados.

3. Envío de la plantilla al motor

Descripción:

Cuando un cliente realiza una solicitud (GET, POST, etc.), el servidor invoca la plantilla correspondiente y le pasa los datos dinámicos.

El motor de plantillas reemplaza los marcadores en la plantilla con los valores proporcionados.



4. Renderización

Descripción:

El motor de plantillas genera el documento final (usualmente HTML) combinando la estructura de la plantilla con los datos dinámicos. Este documento se envía al navegador del cliente.

Ejemplo del proceso de renderización:

Entrada (Plantilla):

Html:

```
<p>Hola, {{ nombre }}!</p>
```

Datos enviados:

Json:

CopiarEditar

```
{ "nombre": "Juan" }
```

Salida:

Html:

```
<p>Hola, Juan!</p>
```

5. Entrega al cliente

Descripción:

El navegador del cliente recibe el documento renderizado. Este documento es interpretado y mostrado como una página web.

6. Interacción con el cliente

Descripción:

Si la página requiere interacción dinámica (como formularios, botones, o datos actualizables), se ejecutan scripts (JavaScript) que pueden enviar nuevas solicitudes al servidor.

El ciclo puede reiniciarse para renderizar nuevas plantillas o actualizar parcialmente la página (usando AJAX o frameworks como React/Vue.js).

7. Actualización o reutilización

Descripción:

Las plantillas son reutilizables. Se pueden modificar para cambiar el diseño, agregar funcionalidades o mejorar la experiencia del usuario sin afectar la lógica del backend.

Si se extienden de una base, basta con actualizar la plantilla base para que los cambios se reflejen en todas las páginas.

Ejemplo de extensión:

Plantilla hija (index.html):

Html:

```
{% extends "base.html" %}
{% block content %}
    <p>Este es el contenido principal.</p>
{% endblock %}
```

Optimización en el ciclo de vida

Uso de caché: Cachear plantillas renderizadas para solicitudes similares.

Carga parcial (AJAX): Renderizar solo partes necesarias para mejorar la velocidad.

Minimización de plantillas: Reducir el peso del HTML generado (compresión).

Control de flujo en plantillas

El control de flujo en plantillas permite manejar la lógica básica como bucles, condicionales y otras estructuras para personalizar el contenido dinámico que se renderiza. Los motores de plantillas ofrecen formas específicas de implementar estas funcionalidades. A continuación, exploramos cómo funciona el control de flujo en plantillas, con ejemplos para distintos motores populares:

1. Estructuras comunes de control de flujo

a) Condicionales

Permiten mostrar contenido basado en condiciones.

b) Bucles

Iteran sobre listas, objetos o colecciones para generar contenido repetitivo.

c) Bloques personalizados

Permiten incluir o extender plantillas según ciertas reglas.

Ejemplo por motor de plantillas

a) Jinja2 (Flask/Django)

Condicionales

html

```
{% if user.is_authenticated %} <p>Hola, {{ user.name }}!</p> {% else %} <p>Por favor, inicia sesión.</p> {% endif %}
```

Bucles

html

```
<ul> {% for item in items %} <li>{{ item }}</li> {% endfor %} </ul>
```

Condicional dentro de un bucle

html

Control de flujo en plantillas

¿Qué es el anidamiento de plantillas?

a) Plantilla Base

Es una plantilla principal que contiene la estructura común, como el encabezado, el pie de página, y otras secciones generales.

b) Plantilla Hija

Es una plantilla que.

utiliza la base y define o sobrescribe partes específicas de la estructura

Beneficios del anidamiento de plantillas

Reutilización: Reduce la duplicación de código.

Mantenimiento: Cambios en la plantilla base se reflejan en todas las páginas.

Organización: Facilita la separación de diseño y contenido.



Renderizado de listas dinámicas

El renderizado de listas dinámicas en plantillas se refiere a la generación automática de contenido basado en colecciones de datos (como listas, arrays o diccionarios). Esto es útil para mostrar elementos repetitivos, como productos, publicaciones de un blog, o usuarios registrados.

Generalidades del renderizado dinámico de listas ¿Cómo funciona?

El servidor prepara los datos:

La lógica del backend obtiene una colección de datos desde una base de datos o una API.

Los datos se pasan a la plantilla:

Se envían como variables para que el motor de plantillas los procese.

La plantilla itera sobre los datos:

Usa bucles para renderizar cada elemento de la colección en un formato definido.

Ejemplos por motor de plantillas

a) Jinja2

(Flask/Django)

Backend (Flask):

```
python
from flask import Flask, render_template app = Flask(__name__)
@app.route('/') def index(): productos = [ {"nombre": "Laptop", "precio": 1200},
{"nombre":
"Teléfono", "precio": 800}, {"nombre": "Tablet", "precio": 600} ] return
render_template('productos.html', productos=productos) if __name__ ==
'__main__': app.run(debug=True)
```

Plantilla (productos.html):

```
html
<!DOCTYPE html> <html lang="es"> <head> <title>Productos</title> </head>
<body> <h1>Lista de Productos</h1> <ul> {% for producto in productos %}
<li>{{ producto.nombre }} - ${{{ producto.precio }}}</li> {% endfor %} </ul>
</body>
</html>
```

Salida Renderizada:

```
html
<h1>Lista de Productos</h1> <ul> <li>Laptop - $1200</li> <li>Teléfono -
$800</li> <li>Tablet - $600</li> </ul>
```

Uso de plantillas para formularios

El uso de plantillas para formularios permite generar formularios dinámicos, reutilizables y estilizados en aplicaciones web. Los motores de plantillas simplifican la integración de datos dinámicos en formularios, como la previsualización de valores, la validación de campos, y la personalización de estilos.

Componentes de un formulario en plantillas

a) Etiquetas de entrada dinámica

Renderizan valores predeterminados o recuperados.

Ejemplo: Rellenar un formulario con datos de usuario existentes.

b) Validación y retroalimentación

Mostrar errores específicos para campos.

Asegurar la validación tanto del lado cliente como del servidor.

c) Estilización dinámica

Aplicar estilos según el estado del campo (vacío, válido, inválido).



Creación de vistas personalizadas

La creación de vistas personalizadas implica desarrollar funcionalidades específicas en tu aplicación que presenten información, gestionen lógica personalizada y proporcionen experiencias dinámicas a los usuarios. Estas vistas pueden ser generadas para tareas como dashboards, páginas de perfil, secciones interactivas o cualquier funcionalidad que necesite un comportamiento exclusivo.

¿Qué es una vista personalizada?

Una **vista** es una función o clase que responde a una solicitud HTTP y genera una respuesta (HTML, JSON, etc.).

Una **vista personalizada** contiene lógica específica que no se puede resolver con vistas genéricas estándar.



Mejorando el rendimiento del renderizado

Mejorar el rendimiento del renderizado en aplicaciones es crucial para ofrecer una experiencia rápida y fluida a los usuarios. Esto incluye optimizaciones tanto del lado del cliente como del servidor para reducir tiempos de carga, minimizar el uso de recursos y mejorar la eficiencia en el procesamiento de datos

Optimización en el lado del servidor

a) Uso de plantillas ligeras

Mantén las plantillas simples y claras.

Evita realizar cálculos o lógica compleja en las plantillas.

Ejemplo:

En lugar de calcular un valor en la plantilla:

html

```
<p>Total: {{ precio_unitario * cantidad }}</p>
```

Hazlo en el backend:

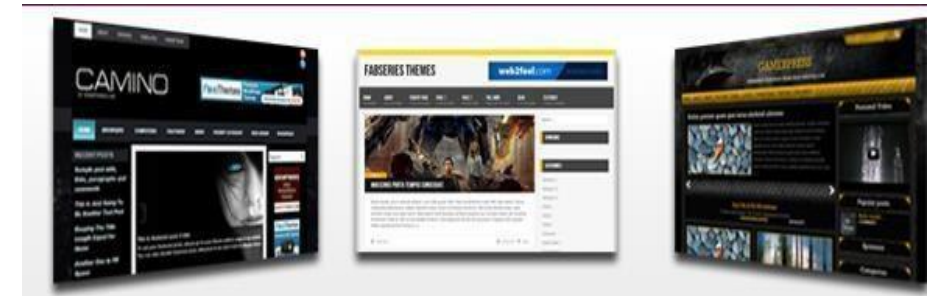
python

```
datos["total"] = precio_unitario * cantidad
```

b) Cacheo de vistas

Almacena las respuestas de vistas estáticas o poco dinámicas.

Usa sistemas como Redis o cacheo en memoria.

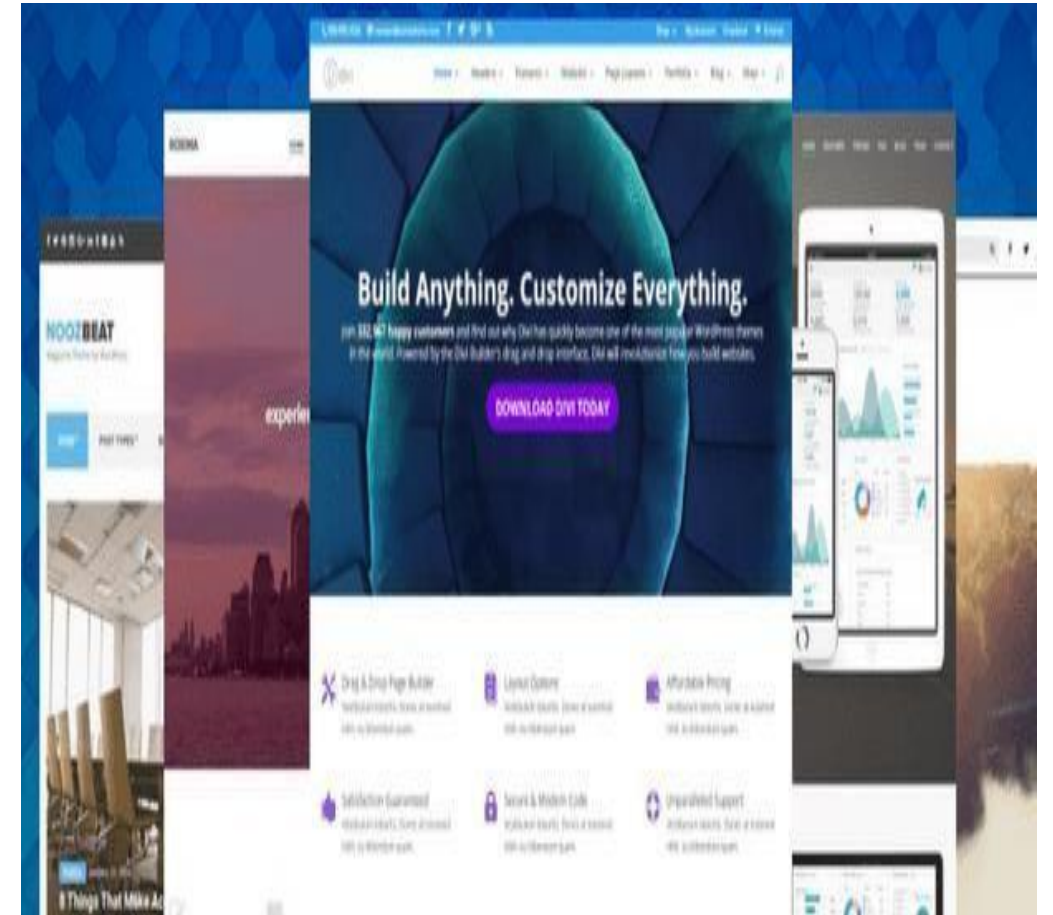


C) Paginación:

- Divide grandes grupos de datos en paginas pequeñas

D) Compresión de respuestas

- Reduce el tamaño de las respuestas HTTP
- Habilitar Gzip o Brotli en tu servidor



Optimización en el lado del cliente

e) Reducir la cantidad de consultas al servidor

Pre-carga o almacenamiento en caché en el cliente.

Ejemplo con IndexedDB (JavaScript):
Javascript

f) Minificación de archivos

- Usa herramientas como **Webpack**, **Terser**, o **PostCSS** para minimizar archivos CSS y JS.
- **Lazy Loading**
- Carga diferida de imágenes o datos cuando son necesarios.

Ejemplo con imágenes:

- html
- ``

g) Uso de frameworks SPA React, Vue o Angular permiten el renderizado parcial de vistas, actualizando solo los componentes necesarios.

Buenas prácticas al usar plantillas

El uso de plantillas en aplicaciones web puede hacer que el desarrollo sea más eficiente y organizado, pero es importante seguir algunas buenas prácticas para mantener el código limpio, seguro y fácil de mantener. Aquí te dejo algunas recomendaciones clave:

Separación de lógica y presentación

a) Mantén la lógica de negocio en el backend.

Las plantillas deben centrarse únicamente en la presentación y no incluir lógica compleja o de negocio. Esto permite que el código sea más modular y fácil de mantener.

Seguridad en el uso de plantillas

a) Prevención de inyecciones XSS

La interpolación de datos dinámicos en las plantillas puede ser vulnerable a ataques XSS si no se escapan adecuadamente los valores. Los motores de plantillas suelen hacer esto automáticamente, pero siempre verifica que se escape el contenido de manera adecuada.

b) Validación de datos de entrada

Aunque las plantillas pueden mostrar datos dinámicos, la validación de los datos debe realizarse en el backend antes de ser enviados a la plantilla.

. Reutilización de componentes

a) Uso de plantillas base

Es una buena práctica tener una plantilla base que defina la estructura común de todas las páginas, como el encabezado, pie de página, y barra de navegación. Las vistas específicas luego extienden esta plantilla.

Manejo de errores y excepciones

a) Muestra mensajes de error de forma amigable

Si hay errores de validación o de otro tipo, muestra mensajes amigables al usuario y asegúrate de que los errores no expongan detalles internos.

Optimización de rendimiento

a) Minimiza el procesamiento en el frontend

Evita realizar cálculos complejos o transformaciones de datos en las plantillas. Si es posible, realiza estos cálculos en el backend y pasa solo los datos necesarios a la plantilla.

b) Cacheo de contenido estático

Cuando trabajes con contenido estático (como imágenes, archivos CSS o JavaScript), asegúrate de implementar el cacheo adecuado para reducir las solicitudes repetidas al servidor.

Uso de condicionales y ciclos de manera eficiente

a) Evita lógica excesiva en las plantillas

Aunque los motores de plantillas permiten usar estructuras como condicionales y ciclos, trata de que estas solo manejen la presentación. La lógica compleja debe estar en el controlador o backend.

Organización de archivos de plantillas

Mantén las plantillas bien estructuradas

Si tu proyecto es grande, organiza las plantillas en carpetas, según su funcionalidad

Prácticas de accesibilidad

Incluye atributos accesibles

Cuando estés diseñando formularios, enlaces, y otros elementos interactivos, asegúrate de que sean accesibles para usuarios con discapacidades.

Evita duplicación de código

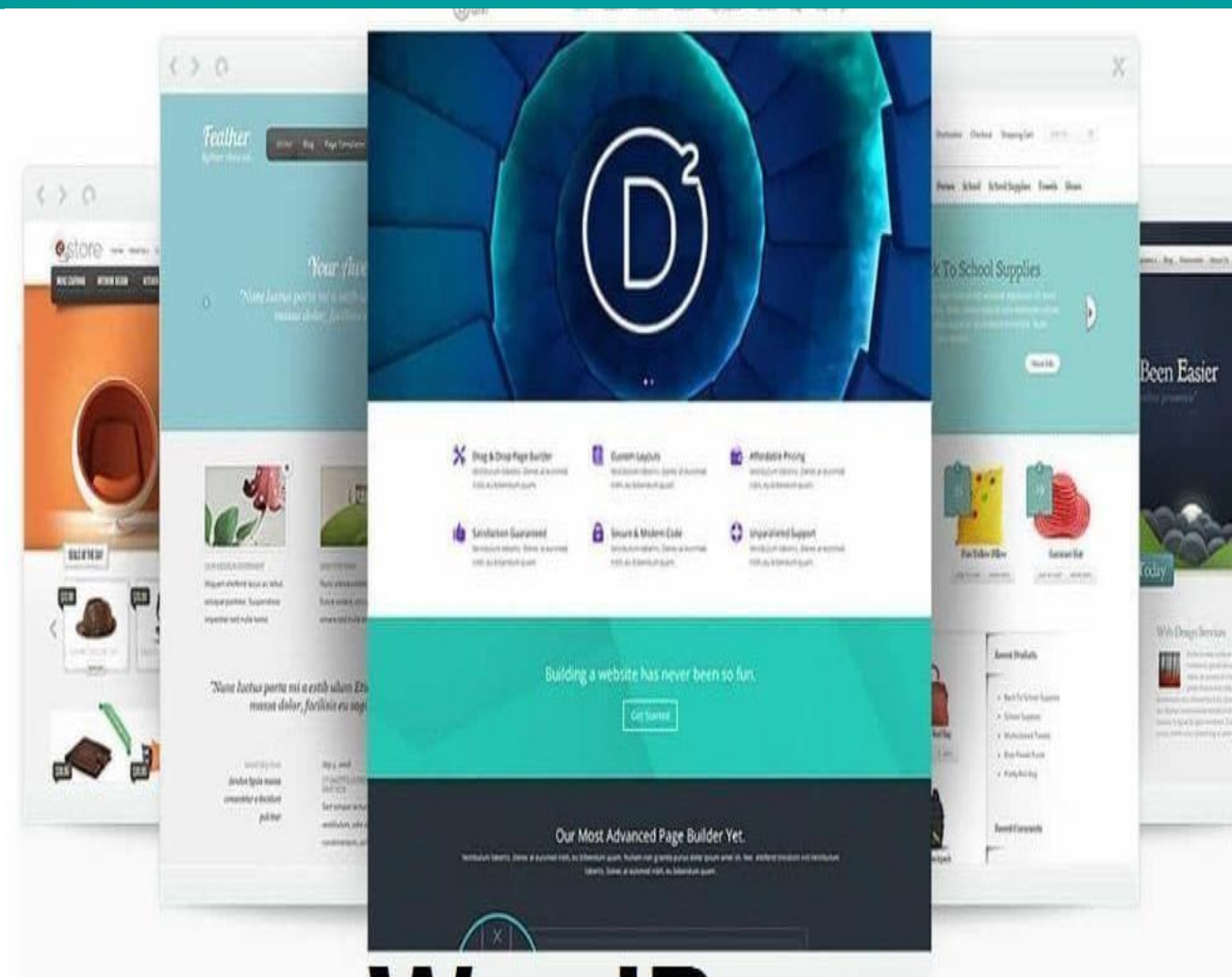
Reutiliza fragmentos

Si tienes componentes comunes (como formularios o tablas), usa fragmentos o includes para evitar repetir código.

Actualización y mantenimiento continuo

Revisión periódica

A medida que la aplicación crece, es importante revisar y actualizar las plantillas para asegurarse de que siguen siendo eficientes, seguras y fáciles de entender.



JUNTOS TRANSFORMAMOS EL MUNDO DESDE LA AMAZONÍA

